



Have you heard of that new band "1023 Megabytes"?
They're pretty good,
but they don't have a gig just yet.

CMPT 295

Unit - Data Representation

Lecture 6 – Representing real numbers in memory – IEEE 754 standard for
floating point representation – Encoding Scheme

Last lecture

- Demo – Impact of fixed size memory on integer additions in **C**
- Encoding schemes **B2U(x)**, **U2B(x)**, **B2T(x)**, **T2B(x)** represent integral numbers we store in memory
 - Can encode a **small** range of integral values **exactly** (in 1, 2, 4, 8 bytes)
 - Example: We can represent the small range of integral values [-128 .. 127] **exactly** in 1 byte using a **signed char** in **C**
- Encoding scheme **IEEE 754 standard for floating point representation** represents real numbers we store in memory
 - Can encode a **much larger** range of real values, but **approximately** (in 4 or 8 bytes)
 - Example: We can represent the large range of real values $[-1.18 \times 10^{-38} \dots +3.4 \times 10^{38}]$ **approximately** in 4 bytes (single precision) using a **float** in **C**
- Overview of **IEEE 754 standard** - **R2B (x)** and **B2R (x)**

Today's menu

- Storing data in memory

- “Under the Hood” - **von Neumann** architecture

- Bits and bytes in memory

Note that we do not know yet what data these bits represent!

- How to diagram memory (used in this course and other references)

- reviewed in Lab 0** ➤ How to express memory content -> **using binary and hexadecimal numeral system**

- Order of bytes in memory -> **endian**

- Bit manipulation – bitwise operations

- Boolean algebra + shifting

We shall start giving meaning to bits -> what data they represent.

- What kind of data do bits (in memory) represent -> **encoding scheme**

- Bits representing (encoding) integral numbers

- reviewed in Lab 1** ➤ Unsigned and signed

- practiced in Lab 2** ➤ Converting, expanding and truncating – Impact of fixed size memory

- Arithmetic operations

- Bits representing (encoding) real numbers

- IEEE floating point representation (cont'd)

- Floating point in C – casting, rounding, addition, ...

Overview of IEEE 754 standard:

R2B (X) and B2R (X)

➡ **R2B (X)** - How to convert a real number (**R**) to an **IEEE 754 floating point number** (i.e., bit pattern **B** ):

✓ Convert the given real number **R** into a binary number (with integral and fractional parts as well as a binary point) using the **positional notation**.

- ➡ 1. Normalize this binary number into the **binary numerical form** $V = (-1)^s M 2^E$
2. Convert this binary number expressed in the **binary numerical form** **V** into an **IEEE 754 floating point number** (i.e., a bit pattern **B**) using the **appropriate** equations.

➡ **B2R (X)** - How to convert an **IEEE 754 floating point number** (**B**) to a real number (**R**):

1. Reverse the above algorithm.

Review: Scientific Notation and normalization

- From Wikipedia:
 - **Scientific notation** is a way of expressing numbers that are too large or too small to be conveniently written in decimal form (as they are long strings of digits).
 - In scientific notation, nonzero numbers are written in the form $\pm M \times 10^n$
 - In **normalized notation**, the exponent n is chosen such that the absolute value of the significand M is at least 1 ($M = 1.0$) but less than **the base**

B range for **base 10** $\Rightarrow [1.0 \dots 10.0 - \epsilon]$
M range for **base 2** $\Rightarrow [1.0 \dots 2.0 - \epsilon]$

- Examples:
 - A proton's mass is 0.00000000000000000000000000000016726 kg -> 1.6726×10^{-27} kg
 - Speed of light is 299,792,458 m/s -> $2.99792,458 \times 10^8$ m/s

Syntax of

normalized notation

$$\pm d_0 \cdot d_{-1} d_{-2} d_{-3} \dots d_{-n} \times b^{\text{exp}}$$

sign

significand

base exponent

Binary numerical form $V = (-1)^s M 2^E$

Just like we can represent real numbers in

➤ **scientific notation** (base 10): $+/- M \times 10^n$

We can represent their floating-point encodings using a base 2 version of the scientific notation:

➤ **binary numerical form:** $V = (-1)^s M 2^E$

➤ **s** -> determines whether number is -ve or +ve

➤ **M** -> Significand (or Mantissa) -> fractional part of number

➤ **E** -> Exponent

➤ Examples:

➤ $V = 1.01011010101_2 \times 2^8$

➤ $V = -1.111101_2 \times 2^5$

Step 2 of **R2B (X)** – Let's normalize the following binary numbers!

1. $101011010.101_2 =$

2. $0.000000001101_2 =$

3. $-11000000111001_2 =$

B2R (X) – Reversing the normalization, i.e.,
denormalized the following binary #'s (**V**)

1. $-1.111101_2 \times 2^5 =$

2. $1.0_2 \times 2^{-8} =$

3. $1.1000000111001_2 \times 2^{-1} =$

Step 3 of **R2B (X)** – First, let's talk about *appropriate* equations

- In Step 3 of **R2B (X)**, we convert a binary number of the normalized form $V = (-1)^s M 2^E$ where ...
 - **s** -> determines whether number is -ve or +ve
 - **M** -> Significand (or Mantissa) -> fractional part of number
 - **E** -> Exponent

into an **IEEE 754 floating point number**, i.e., a bit pattern of the form:



where ...

- The most significant bit (MSb) **s** encodes the sign
- **exp** field encodes **E** (but *is not equal* to **E**)
- **frac** field encodes **M** (without the binary point ".")

using the *appropriate* equations.

Let's define these **appropriate** equations ...

Starting with the **Normalized** range

- $(-1)^s$ -> If real number is
- positive => $s = 0$
 - negative => $s = 1$

Binary numerical form: $V = (-1)^s M 2^E$



Normalized: $\text{exp} \neq 0$ and $\text{exp} \neq 11\dots 11$
(exp range: $[00000001 \dots 11111110]$)

Equations:

$$E = \text{exp} - \text{bias} \text{ and } \text{bias} = 2^{k-1} - 1$$

$$M = 1. + \text{frac}$$

Why using a **bias**?

Why adding **1.** to **frac**?

Here is why we use a **bias**!

Binary numerical form: $V = (-1)^s M 2^E$



...

Equations:

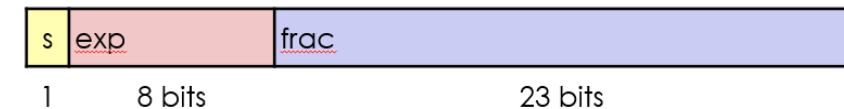
$$E = \text{exp} - \text{bias}$$

$$\text{and } \text{bias} = 2^{k-1} - 1$$

$$M = 1. + \text{frac}$$

Why is **E** biased?

Using **single precision** as an example:



- **exp** range: $[00000001 \dots 11111110] \Rightarrow [1_{10} \dots 254_{10}]$
- If **E is not biased** (i.e., $E = \text{exp}$), then **E** range: $[1_{10} \dots 254_{10}]$
- 2^E range: $[2^1 \dots 2^{254}] = [2.0 \dots \sim 2.89 \times 10^{76}]$
- By **biasing E** (i.e., $E = \text{exp} - \text{bias}$), then **E** range: $[1_{10} - 127 \dots 254_{10} - 127]$
 $= [-126 \dots 127]$ (since $k = 8$, $\text{bias} = 2^{8-1} - 1 = 127$)
- 2^E range: $[2^{-126} \dots 2^{127}] = [\sim 1.18 \times 10^{-38} \dots \sim 1.7 \times 10^{38}]$

so cannot express numbers < 2 ☹

so can now express very small (and large) numbers ☺

Here is why we add 1. to **frac**!

Binary numerical form: $V = (-1)^s M 2^E$



...

Equations:

$$E = \text{exp} - \text{bias}$$

$$\text{and } \text{bias} = 2^{k-1} - 1$$

$$M = 1. + \text{frac}$$

Why adding 1. to **frac**?

We add 1. to **frac** because the value **V** is in a **normalized form** and all normalized binary numbers **V** start with 1.

Obtained once we convert the real number into a binary number using the positional notation

Why adding 1. to **frac**? Revisited

- A binary number is first normalized (i.e. expressed in the form **V**) before it is converted.
- As part of this normalization process, we transform our binary number such that its significand **M** is within the range $[1.0 \dots 1.11111\dots]$
- Because $1.11111\dots$ is infinite, it is expressed as **2.0** - ϵ
- So, **M** range for **base 2** $\Rightarrow [1.0 \dots \mathbf{2.0} - \epsilon]$
- This implies that **M** is **always** at least **1.0**, at most **1.11111...**, so its *integral* part always has the value 1 followed by "." (binary point)
- So since this bit is always part of **M**, IEEE 754 standard does not explicitly save it in its bit pattern (i.e., in memory)
- Instead, this bit is implied and so is the binary point "."

Why adding 1. to **frac**? Revisited (cont'd)

We get the leading bit for free!

Implying this bit has the following effects:

1. We **save 1 bit** when we convert (represent) a real number into a bit pattern using IEEE 754 standard
2. We have to add this 1 bit (and the binary point ".") back when we convert from a bit pattern (**B**) (IEEE 754 floating point number) back to a real number (**R**)

Example: $V = (-1)^s M 2^E = 1.01011010101 \times 2^8$

$M = 1.01011010101 \Rightarrow M = 1. + \text{frac}$

↑
This bit is implied hence not stored in the bit pattern (**B**) produced by the IEEE 754 standard for floating point representation, so what we store in the **frac** part of the IEEE 754 standard bit pattern is
01011010101

The binary point is also implied and not represented in the bit pattern

Step 3 of **R2B (X)** – Let's practice this step!

(single precision => **k** = 8 bits, **n** => 23 bits)

Imagine Step 1 and 2
have already been done!

3. Considering a binary number in form **V**,
we apply the **equations**

$$\Rightarrow \mathbf{V} = (-1)^{\mathbf{s}} \mathbf{M} 2^{\mathbf{E}} = 1.01011010101_2 \times 2^8$$

$$\Rightarrow \mathbf{s} =$$

$$\Rightarrow \mathbf{E} = \mathbf{exp} - \mathbf{bias} \text{ where } \mathbf{bias} = 2^{k-1} - 1 = 2^7 - 1 = 128 - 1 = 127$$

$$\mathbf{exp} = \mathbf{E} + \mathbf{bias} =$$

$$\Rightarrow \mathbf{M} = 1. + \mathbf{frac} =$$

bit pattern
in memory:



k bits => 8 bits

n bits => 23 bits

$\Rightarrow$ in **hex**:

Normalized:

exp $\neq 0$ and **exp** $\neq 11\dots 11$
(**exp** range:
[00000001 .. 11111110])

Equations:

E = **exp** - **bias** and **bias**
 $= 2^{k-1} - 1$

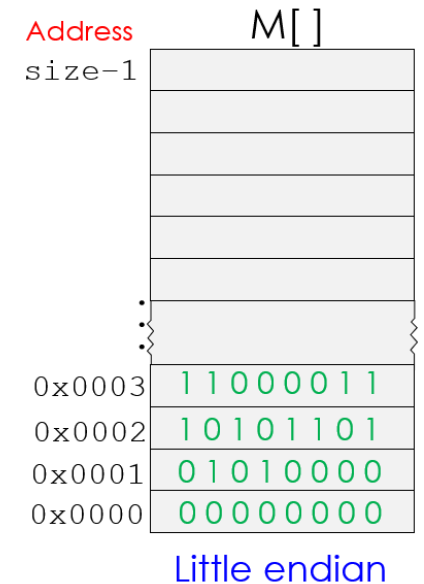
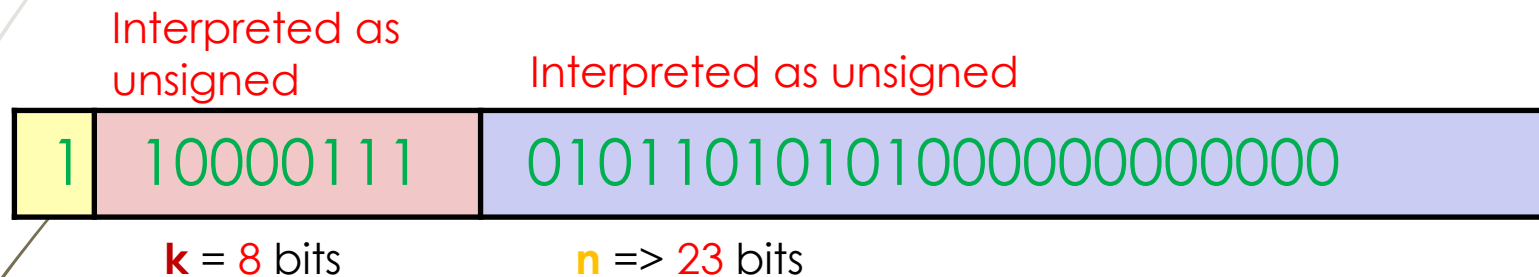
M = 1. + **frac**

(single precision => **k** = 8 bits, **n** => 23 bits)

(single precision => **k** = 8 bits, **n** => 23 bits)

1. What if the 4 bytes starting at $M[0 \times 0000]$ represents a real number (encoded as a single precision IEEE 754 floating point number), what would its value be?

Binary numerical form: $V = (-1)^s M 2^E$



- **exp** within range: [00000001 .. 11111110] so -> **normalized**
- **s** =
- **E** = **exp** - **bias** where **bias** = $2^{k-1} - 1 = 2^7 - 1 = 128 - 1 = 127$
- **E** = _____ - 127 =
- **M** = 1. + **frac** = 1 + _____
- **V** =

Let's practice all 3 steps of **R2B (X)** ! (single precision)

How would **47.21875** be encoded as IEEE 754 floating point number?

1. Convert **47.21875** to binary (using the **positional notation**)

$$\Rightarrow \mathbf{47} = 101111_2$$

$$\Rightarrow \mathbf{.21875} = .00111_2$$

2. Normalize the resulting binary number:

$$101111.00111 \Rightarrow 1.0111100111_2 \times 2^5$$

Binary numerical form: $\mathbf{V} = (-1)^s \mathbf{M} 2^E$

3. Determine ...

$$\mathbf{s} = 0$$

$$\mathbf{E} = \mathbf{exp} - \mathbf{bias} \text{ where } \mathbf{bias} = 2^{k-1} - 1 = 2^7 - 1 = 128 - 1 = 127$$

$$\mathbf{exp} = \mathbf{E} + \mathbf{bias} = 5 + 127 = 132 \Rightarrow \mathbf{U2B(132)} \Rightarrow 10000100_2$$

$$\mathbf{M} = 1. + \mathbf{frac} \rightarrow \mathbf{frac} = \mathbf{M} - 1. \Rightarrow 1.0111100111_2 - 1. = 0111100111_2$$



$\Rightarrow$ In **hex**: 0x423CE000

Let's practice all 3 steps of **B2R (X)** ! (single precision)

Which real number does **0xBF950000** (**B**) represent on our *target machine*?

1. **0xBF950000** ->



Determine ...

s =

E = **exp** - **bias** = - 127 =

M = 1. + **frac** =

Binary numerical form: **V** =

$$\mathbf{V} = (-1)^{\mathbf{s}} \mathbf{M} 2^{\mathbf{E}}$$

2. Denormalize **V**:

3. Using the **positional notation**, convert the above denormalized binary number into **R**:

Homework

Let's practice all 3 steps of **R2B (X)** ! (single precision)

How would **12345.75** be encoded as IEEE 754 floating point number?

1. Convert 12345.75 to binary

➡ 12345 => .75 =>

2. Normalize binary number:

Binary numerical form: $V = (-1)^s M 2^E$

3. Determine ...

s =

E = **exp** - **bias** where **bias** = $2^{k-1} - 1 = 2^7 - 1 = 128 - 1 = 127$

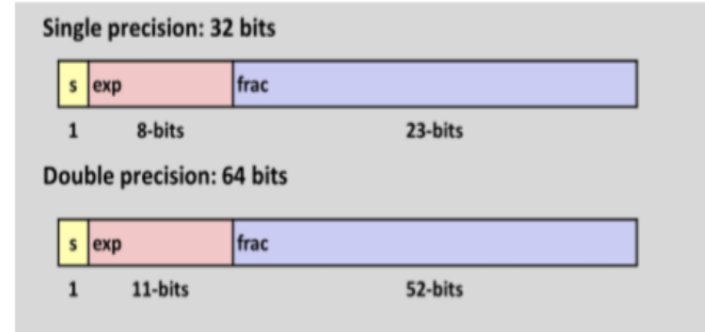
exp = **E** + **bias** =

M = 1. + **frac** -> **frac** = **M** - 1.



In **hex**:

Summary



➤ IEEE 754 standard - R2B (X) :

1. Convert the given real number **R** into a binary number (with integral and fractional parts as well as a binary point) using the **positional notation**.

2. Normalize this binary number into the **binary numerical form**

$$V = (-1)^s M 2^E$$

3. Convert this binary number expressed in the **binary numerical form V** into an **IEEE 754 floating point number** (i.e., a bit pattern **B**) using the **appropriate** equations.

➤ **Normalized** => **exp** within range: [00000001 .. 11111110]

➤ **E** = **exp** - **bias** where **bias** = $2^{k-1} - 1$

➤ **M** = 1. + **frac**

➤ IEEE 754 standard - B2R (X) :

1. Reverse the above algorithm.

Next Lecture

- Storing data in memory

- “Under the Hood” - **von Neumann** architecture

- Bits and bytes in memory

Note that we do not know yet what data these bits represent!

- How to diagram memory (used in this course and other references)

- reviewed in Lab 0** ➤ How to express memory content -> **using binary and hexadecimal numeral system**

- Order of bytes in memory -> **endian**

- Bit manipulation – bitwise operations

- Boolean algebra + shifting

We shall start giving meaning to bits -> what data they represent.

- What kind of data do bits (in memory) represent -> **encoding scheme**

- Bits representing (encoding) integral numbers

- reviewed in Lab 1** ➤ Unsigned and signed

- practiced in Lab 2** ➤ Converting, expanding and truncating – Impact of fixed size memory

- Arithmetic operations

- Bits representing (encoding) real numbers

- IEEE floating point representation (cont'd)

- Floating point in C – casting, rounding, addition, ...